جزوهٔ مطالعه

جلسهٔ ۶: طراحی شیءگرا و SOLID

مهندسی نرم‌افزار

تهیه‌کننده: ابراهیم فضلی

ترم: ۱۴۰۴

۹ آبان ۱۴۰۵

درس	مهندسی نرم‌افزار
جلسه	۶
موضوع	طراحی شیءگرا، SOLID، Coupling/Cohesion
زمان مطالعه	۳-۴ ساعت
پیش‌نیاز	جلسات ۱-۵

فهرست مطالب

۱ مقدمه

۱.۱ چرا طراحی شیءگرا مهم است؟

در جلسات قبل یاد گرفتیم:

- جلسهٔ ۱: مهندسی نرم‌افزار چیست
- جلسهٔ ۲: نیازمندی‌ها
- جلسهٔ ۳: فرآیندهای توسعه
- جلسهٔ ۴: معماری و مدل‌سازی
- جلسهٔ ۵: Review Sprint

حالا وقت آن است که کد بهتر بنویسیم.

۲.۱ مشکل کد بد

تصور کنید یک کلاس Order دارید که:

- مجموع را محاسبه می‌کند
- در دیتابیس ذخیره می‌کند
- ایمیل تأیید می‌فرستد
- فاکتور چاپ می‌کند
- به انبار اطلاع می‌دهد

مشکل چیست؟

- اگر روش ارسال ایمیل عوض شود، باید Order را تغییر دهیم
- اگر دیتابیس عوض شود، باید Order را تغییر دهیم
- تست کردن سخت است
- استفادهٔ مجدد ممکن نیست

نتیجه: کد بد = هزینهٔ تغییر بالا.

۳.۱ راه‌حل: طراحی شیءگرا

طراحی شیءگرا به ما کمک می‌کند:

- کد را به بخش‌های کوچک تقسیم کنیم
- هر بخش یک مسئولیت داشته باشد
- تغییرات، کم‌ترین تأثیر را داشته باشند
- تست کردن آسان باشد

۴.۱ آنچه در این جزوه یاد می‌گیرید

- چهار اصل OOP
- پنج اصل SOLID
- Coupling و Cohesion
- Patterns Design (مقدمه)
- چطور کد بد را Refactor کنیم

۲ چهار اصل OOP

۱.۲ Encapsulation – کپسوله‌سازی

۱.۱.۲ تعریف

داده‌ها و متدها در یک واحد (کلاس) قرار می‌گیرند. داده‌های داخلی پنهان هستند و فقط از طریق متدهای عمومی قابل دسترسی‌اند.

۲.۱.۲ مثال بد

```
{ tcudorP ssalc cilbup
;eman gnirtS cilbup
;ecirp elbuod cilbup
;kcots tni cilbup
}
```

مشکل: هر کسی می‌تواند price را منفی کند.

۳.۱.۲ مثال خوب

```
{ tcudorP ssalc cilbup
;eman gnirtS etavirp
;ecirp elbuod etavirp
;kcots tni etavirp

{ )ecirp elbuod(ecirPtes diov cilbup
{ )0 < ecirp( fi
eb tonnac ecirP"(noitpecxEtnemugrAlagellI wen worht
```

```
                ;)"evitagen
                 }
          ;ecirp = ecirp.siht
                   }
        { )(ecirPteg elbuod cilbup
            ;ecirp nruter
                   }
                 }
```

مزیت: قیمت منفی ممکن نیست.

۲.۲ Inheritance – ارث‌بری

۱.۲.۲ تعریف

کلاس فرزند، ویژگی‌ها و متدهای کلاس والد را به ارث می‌برد.

۲.۲.۲ مثال

```
                { resU ssalc cilbup
            ;liame gnirtS detcetorp
            ;eman gnirtS detcetorp
          } ... { )(nigol diov cilbup
                }
      { resU sdnetxe nimdA ssalc cilbup
    } ... { )(stcudorPeganam diov cilbup
                }
```

۳.۲.۲ مزیت و عیب

مزیت: استفادهٔ مجدد از کد.

عیب: وابستگی زیاد. تغییر در User ممکن است Admin را بشکند.

توصیه: ترکیب (Composition) بهتر از ارث‌بری (Inheritance) است.

۳.۲ Polymorphism – چندشکلی

۱.۳.۲ تعریف

یک نام، چند رفتار. متد مشترک، پیاده‌سازی متفاوت.

۲.۳.۲ مثال

```
{ dohteMtnemyaP ecafretni cilbup
  ;)tnuoma elbuod(yap diov
}

{ dohteMtnemyaP stnemelpmi tnemyaPdraCtiderC ssalc cilbup
} ... { )tnuoma elbuod(yap diov cilbup
}

{ dohteMtnemyaP stnemelpmi tnemyaPlaPyaP ssalc cilbup
} ... { )tnuoma elbuod(yap diov cilbup
}

//
;)(tnemyaPdraCtiderC wen = tnemyap dohteMtnemyaP
;)000001(yap.tnemyap
```

مزیت: کد عمومی، رفتار خاص.

۴.۲ Abstraction – انتزاع

۱.۴.۲ تعریف

پنهان کردن جزئیات پیاده‌سازی. کاربر فقط چه کاری انجام می‌دهد را می‌بیند.

۲.۴.۲ مثال

```
{ yrotisopeRredrO ecafretni cilbup
  ;)di tni(dIyBdnif redrO
  ;)redro redrO(evas diov
}
```

```
)(evas          //
```

ابزار: Class. Abstract Interface،

۳ SOLID – پنج اصل طراحی

۱.۳ (SRP) Principle Responsibility Single – S

۱.۱.۳ تعریف

> هر کلاس فقط یک مسئولیت داشته باشد. فقط یک دلیل برای تغییر.

۲.۱.۳ مثال بد

```
{ redrO ssalc cilbup
} ... { )(latoTetaluclac diov cilbup
} ... { )(evas diov cilbup
} ... { )(liamEdnes diov cilbup
} ... { )(eciovnItnirp diov cilbup
}
```

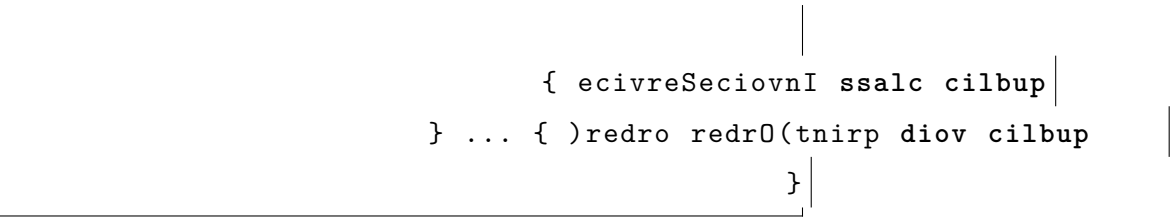
مشکل: ۴ دلیل برای تغییر.

۳.۱.۳ مثال خوب

```
{ redrO ssalc cilbup
} ... { )(latoTetaluclac diov cilbup
}

{ yrotisopeRredrO ssalc cilbup
} ... { )redro redrO(evas diov cilbup
}

{ ecivreSliamE ssalc cilbup
} ... { )redro redrO(noitamrifnoCdnes diov cilbup
}
```



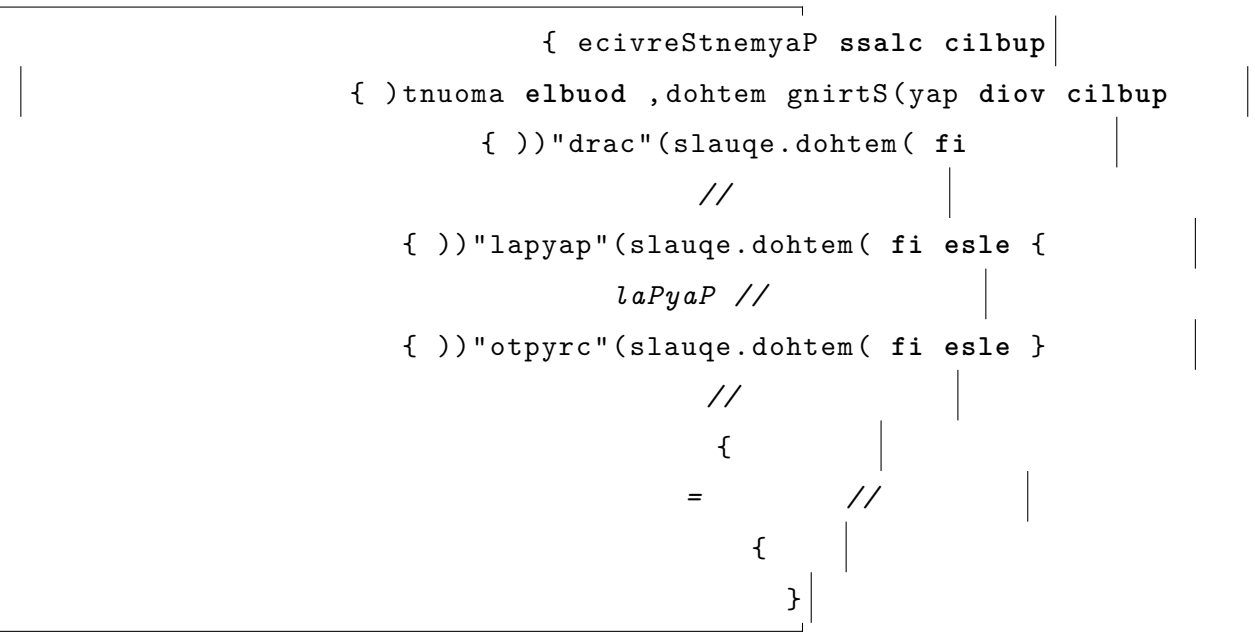
مزیت: هر کلاس یک مسئولیت.

۲.۳ (OCP) Principle Open/Closed – O

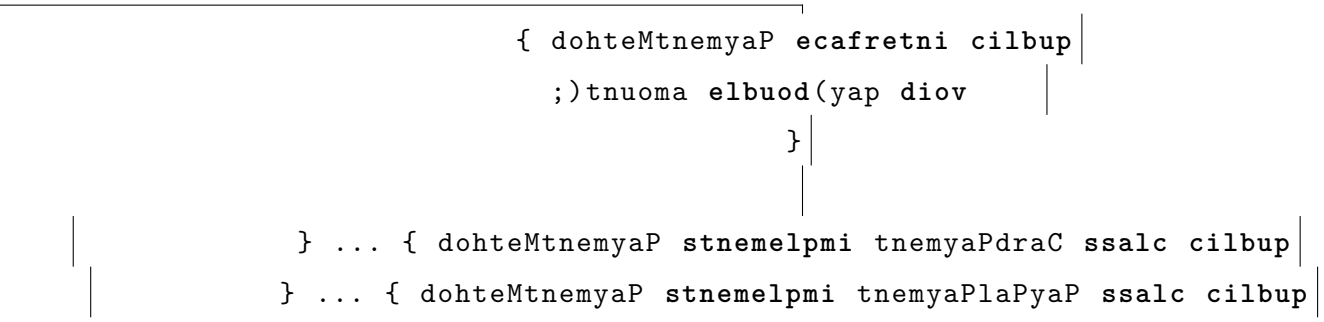
۱.۲.۳ تعریف

> باز برای توسعه، بسته برای تغییر.

۲.۲.۳ مثال بد



۳.۲.۳ مثال خوب



```
                                        |
                                =       //|
    } ... { dohteMtnemyaP stnemelpmi tnemyaPotpyrC ssalc cilbup|
```

مزیت: افزودن روش جدید بدون تغییر کد قدیمی.

۳.۳ L – Liskov Substitution Principle (LSP)

۱.۳.۳ تعریف

> فرزند باید بتواند جایگزین والد شود، بدون شکستن رفتار.

۲.۳.۳ مثال بد

```
                                { elgnatceR ssalc cilbup|
                            ;thgieh ,htdiw tni detcetorp |
                    } ;w = htdiw { )w tni(htdiWtes diov cilbup |
                } ;h = thgieh { )h tni(thgieHtes diov cilbup |
                                }|
                                |
                { elgnatceR sdnetxe erauqS ssalc cilbup|
                    { )w tni(htdiWtes diov cilbup |
                            ;w = htdiw |
                    PSL    // ;w = thgieh |
                                {  |
                                }|
```

مشکل: اگر کسی Rectangle را با Square جایگزین کند، رفتار عوض می‌شود.

۳.۳.۳ مثال خوب

```
                                { epahS ecafretni cilbup|
                                    ;)(aera tni |
                                }|
                                |
                } ... { epahS stnemelpmi elgnatceR ssalc cilbup|
                  } ... { epahS stnemelpmi erauqS ssalc cilbup|
```

مزیت: هر دو مستقل، هیچ‌کدام رفتار دیگری را نقض نمی‌کند.

۴.۳ I – Interface Segregation Principle (ISP)

۱.۴.۳ تعریف

> کلاس نباید متدهای غیرضروری را پیاده کند.

۲.۴.۳ مثال بد

```
{ yawetaGtnemyaP ecafretni cilbup
   ;)tnuoma elbuod(yap diov
  ;)tnuoma elbuod(dnufer diov
   ;)di tni(sutatSkcehc diov
                          }

                          //
{ yawetaGtnemyaP stnemelpmi tnemyaPelpmiS ssalc cilbup
      } ... { )tnuoma elbuod(yap diov cilbup
        { )tnuoma elbuod(dnufer diov cilbup
  ;)(noitpecxEnoitarepOdetroppusnU wen worht
                          }
         { )di tni(sutatSkcehc diov cilbup
  ;)(noitpecxEnoitarepOdetroppusnU wen worht
                          }
                        }
```

۳.۴.۳ مثال خوب

```
{ yawetaGtnemyaP ecafretni cilbup
   ;)tnuoma elbuod(yap diov
                        }

 { yawetaGdnufeR ecafretni cilbup
  ;)tnuoma elbuod(dnufer diov
                        }
```

```
                                    |
                    { yawetaGsutatS ecafretni cilbup|
                      ;)di tni(sutatSkcehc diov     |
                                    }|
```

مزیت: هر کلاس فقط چیزی که لازم دارد را پیاده می‌کند.

۵.۳ D – Principle Inversion Dependency (DIP)

۱.۵.۳ تعریف

> به انتزاع وابسته باش، نه پیاده‌سازی.

۲.۵.۳ مثال بد

```
                                    { ecivreSredrO ssalc cilbup|
                                ;oper yrotisopeRredrOsergtsoP etavirp|
                                    |
                                    { )(ecivreSredrO cilbup     |
                                // ;)(yrotisopeRredrOsergtsoP wen = oper|
                                        {     |
                                          }|
```

مشکل: تغییر دیتابیس = تغییر OrderService.

۳.۵.۳ مثال خوب

```
                                    { ecivreSredrO ssalc cilbup|
                                ;oper yrotisopeRredrOI etavirp|
                                    |
                            { )oper yrotisopeRredrOI(ecivreSredrO cilbup|
                                ;oper = oper.siht     |
                                        }     |
                                      }|
                                      |
                                       //|
                    ;)(yrotisopeRredrOsergtsoP wen = oper yrotisopeRredrOI|
```

```
;)oper(ecivreSredrO wen = ecivres ecivreSredrO
```

مزیت: تغییر دیتابیس = فقط تغییر repo در زمان ساخت.

۴ SRP در HomeStore — مثال واقعی

۱.۴ کلاس بد

```
{ redrO ssalc cilbup
;smeti >metIredrO<tsiL etavirp
;remotsuc remotsuC etavirp

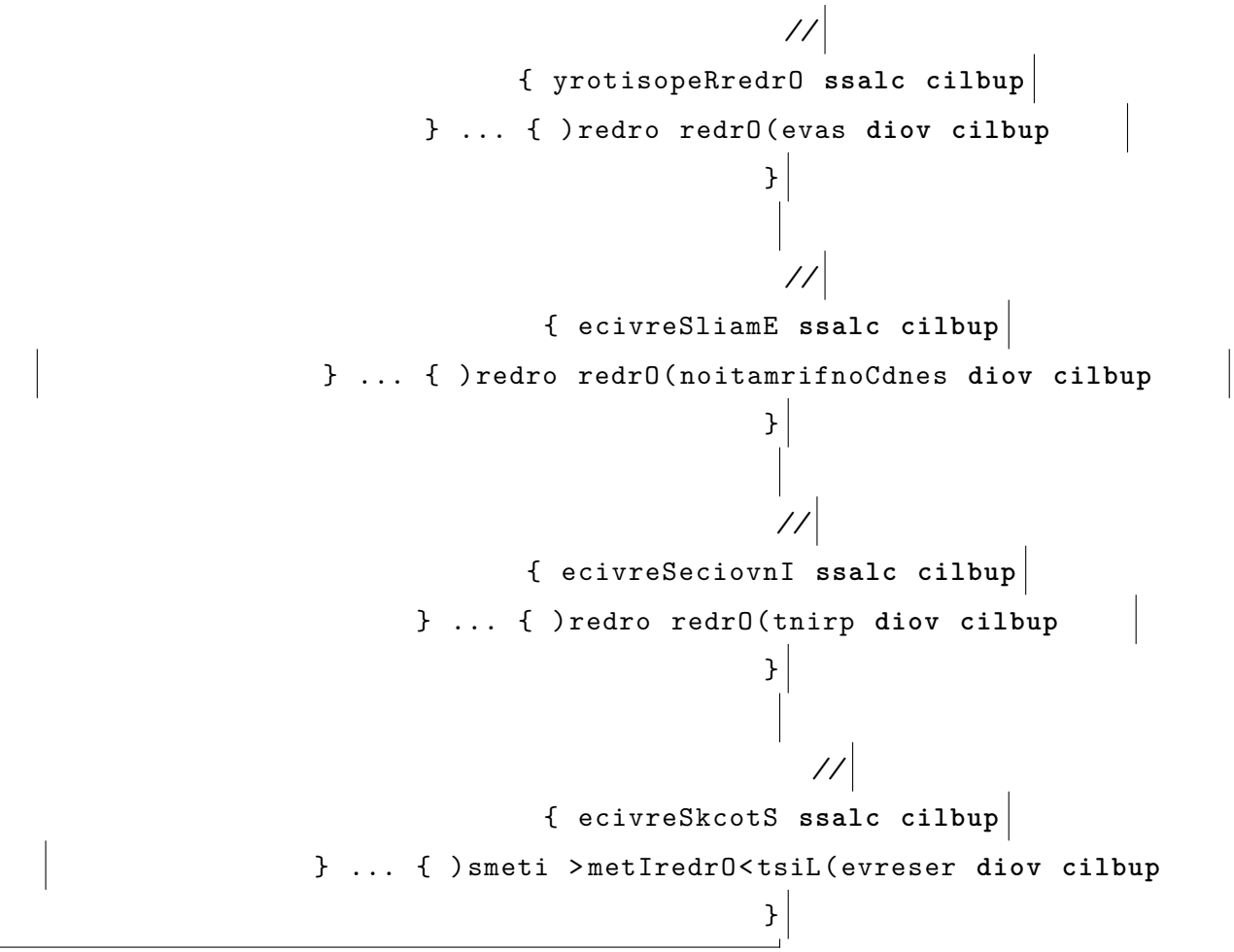
} ... { )(latoTetaluclac elbuod cilbup
} ... { )(evas diov cilbup
} ... { )(liamEdnes diov cilbup
} ... { )(eciovnItnirp diov cilbup
} ... { )(esuoheraWyfiton diov cilbup
}
```

مسئولیت‌ها: محاسبه، ذخیره، ایمیل، چاپ، اطلاع.
دلیل تغییر: ۵ تا!

۲.۴ Refactor شده

```
//
{ redrO ssalc cilbup
;smeti >metIredrO<tsiL etavirp

{ )(latoTetaluclac elbuod cilbup
)(maerts.smeti nruter
)latotbuSteg::metIredrO(elbuoDoTpam.
;)(mus.
}
}
```



۳.۴ نتیجه

- Order فقط محاسبه
- OrderRepository فقط ذخیره
- EmailService فقط ایمیل
- InvoiceService فقط چاپ
- StockService فقط انبار

هر کلاس = یک مسئولیت.

۵ Coupling و Cohesion

۱.۵ تعریف

مفهوم	توضیح
Coupling	وابستگی بین کلاس‌ها — کم بهتر
Cohesion	ارتباط داخلی کلاس — زیاد بهتر

۲.۵ Coupling کم

Coupling کم: تغییر یک کلاس، بقیه را نشکند.

مثال بد:

```
{ ecivreSredrO ssalc cilbup
{ )(ssecorp diov cilbup
;)(yrotisopeRredrOsergtsoP wen = oper yrotisopeRredrOsergtsoP
;)(evas.oper
}
}
```

Coupling بالا: OrderService به PostgresOrderRepository وابسته است.

مثال خوب:

```
{ ecivreSredrO ssalc cilbup
;oper yrotisopeRredrOI etavirp

{ )oper yrotisopeRredrOI(ecivreSredrO cilbup
;oper = oper.siht
}
}
```

Coupling پایین: OrderService به interface وابسته است.

۳.۵ Cohesion زیاد

Cohesion زیاد: همهٔ متدهای کلاس، یک هدف.

مثال بد:

```
{ seitilitU ssalc cilbup
} ... { )(liamEdnes diov cilbup
} ... { )(xaTetaluclac diov cilbup
} ... { )(etaDtamrof diov cilbup
} ... { )(enohPetadilav diov cilbup
}
```

Cohesion کم: ۴ هدف مختلف.

مثال خوب:

```
{ ecivreSliamE ssalc cilbup
} ... { )(dnes diov cilbup
} ... { )(kluBdnes diov cilbup
} ... { )(etadilav diov cilbup
}

{ rotaluclaCxaT ssalc cilbup
} ... { )(etaluclac elbuod cilbup
} ... { )(TAVetaluclac elbuod cilbup
}
```

Cohesion زیاد: هر کلاس یک هدف.

۶ مقدمه — Patterns Design

۱.۶ تعریف

> راه‌حل‌های آماده برای مشکلات رایج طراحی.

۲.۶ دسته‌ها

دسته	مثال
Creational	Builder Singleton، Factory،
Structural	Facade Decorator، Adapter،
Behavioral	Command Observer، Strategy،

۳.۶ مثال: Factory

```
{ dohteMtnemyaP ecafretni cilbup
  ;)tnuoma elbuod(yap diov
}

{ yrotcaFtnemyaP ssalc cilbup
  { )epyt gnirtS(etaerc dohteMtnemyaP citats cilbup
    ;)(tnemyaPdraC wen nruter ))"drac"(slauqe.epyt( fi
    ;)(tnemyaPlaPyaP wen nruter ))"lapyap"(slauqe.epyt( fi
    ;)(noitpecxEtnemugrAlagellI wen worht
  }
}

//
;)"drac"(etaerc.yrotcaFtnemyaP = tnemyap dohteMtnemyaP
;)000001(yap.tnemyap
```

مزیت: کلاینت نمی‌داند کدام کلاس ساخته می‌شود.

۴.۶ مثال: Strategy

```
{ ygetartSgnippihS ecafretni cilbup
;)thgiew elbuod(etaluclac elbuod
}

{ ygetartSgnippihS stnemelpmi gnippihStsoP ssalc cilbup
} ;0005 * thgiew nruter { )thgiew elbuod(etaluclac elbuod cilbup
}

{ ygetartSgnippihS stnemelpmi gnippihSsserpxE ssalc cilbup
} ;00051 * thgiew nruter { )thgiew elbuod(etaluclac elbuod cilbup
}

//
;)(gnippihSsserpxE wen = ygetarts ygetartSgnippihS
;)5.2(etaluclac.ygetarts = tsoc elbuod
```

مزیت: افزودن روش جدید بدون تغییر کد.

۷ سؤالات کلیدی برای مطالعه

۱.۷ سطح ۱ (پایه)

۱.۱.۷ سؤال ۱: Encapsulation چیست؟

پاسخ: کپسوله‌سازی داده و متدها در یک واحد (کلاس) است. داده‌های داخلی پنهان هستند و فقط از طریق متدهای عمومی قابل دسترسی‌اند. مزیت: تغییر داخلی، بدون تأثیر بیرونی.

۲.۱.۷ سؤال ۲: تفاوت Inheritance و Composition چیست؟

پاسخ:
- Inheritance (ارث‌بری): کلاس فرزند از والد ارث می‌برد. وابستگی زیاد.
- Composition (ترکیب): کلاس از کلاس دیگر استفاده می‌کند. انعطاف بیشتر.

توصیه: ترکیب بهتر از ارث‌بری است.

۳.۱.۷ سؤال ۳: SOLID چیست؟

پاسخ: پنج اصل طراحی شیءگرا:
- Responsibility Single S:
- Open/Closed O:
- Substitution Liskov L:
- Segregation Interface I:
- Inversion Dependency D:

۴.۱.۷ سؤال ۴: SRP چیست؟

پاسخ: هر کلاس فقط یک مسئولیت داشته باشد. فقط یک دلیل برای تغییر. مثال: کلاس Order نباید هم محاسبه کند، هم ذخیره، هم ایمیل بفرستد.

۵.۱.۷ سؤال ۵: Coupling و Cohesion چیست؟

پاسخ:
- Coupling: وابستگی بین کلاس‌ها — کم بهتر

• :Cohesion ارتباط داخلی کلاس – زیاد بهتر

۲.۷ سطح ۲ (متوسط)

۱.۲.۷ سؤال ۶: چطور SRP را در کد تشخیص دهیم؟

پاسخ: اگر کلاس بیش از یک دلیل برای تغییر دارد، SRP نقض شده. سؤال بپرسید: «چند نفر ممکن است بخواهند این کلاس را تغییر دهند؟» اگر بیش از یکی، SRP نقض است.

۲.۲.۷ سؤال ۷: تفاوت OCP و DIP چیست؟

پاسخ:
• :OCP باز برای توسعه، بسته برای تغییر. افزودن ویژگی جدید بدون تغییر کد قدیمی.
• :DIP به انتزاع وابسته باش، نه پیاده‌سازی.
هر دو از interface استفاده می‌کنند، ولی هدف متفاوت است.

۳.۲.۷ سؤال ۸: چرا LSP مهم است؟

پاسخ: اگر فرزند نتواند جایگزین والد شود، کدی که با والد کار می‌کند، با فرزند می‌شکند. این باعث باگ‌های پنهان می‌شود.

۴.۲.۷ سؤال ۹: چطور Coupling را کم کنیم؟

پاسخ:
• به interface وابسته باش، نه پیاده‌سازی
• Injection Dependency
• Event-driven
• لایه‌بندی درست

۵.۲.۷ سؤال ۱۰: Pattern Design چیست؟

پاسخ: راه‌حل آماده برای مشکلات رایج طراحی. سه دسته: ،Creational ،Structural .Behavioral

۳.۷ سطح ۳ (پیشرفته)

۱.۳.۷ سؤال ۱۱: اگر کلاسی هم SRP را نقض کند، هم ،DIP اول کدام را درست کنیم؟

پاسخ: اول .SRP چون کلاس بزرگ را به کلاس‌های کوچک تقسیم می‌کنیم، بعد وابستگی‌ها ساده‌تر می‌شوند. بعد DIP را روی کلاس‌های کوچک اعمال می‌کنیم.

۲.۳.۷ سؤال ۱۲: چطور مطمئن شویم Refactor کد را نشکسته؟

پاسخ:
- قبل از Refactor تست بنویس
- بعد از Refactor تست‌ها را اجرا کن
- اگر تست‌ها پاس شدند، Refactor درست است

این اصل Refactor» با تست» است.

۳.۳.۷ سؤال ۱۳: آیا همیشه باید SOLID را رعایت کنیم؟

پاسخ: نه. برای پروژه‌های کوچک، SOLID ممکن است پیچیدگی اضافه کند. SOLID برای پروژه‌هایی است که طول عمر دارند و تغییر می‌کنند. برای اسکریپت‌های یک‌بارمصرف، لازم نیست.

۴.۳.۷ سؤال ۱۴: چطور SOLID را در تیم اجرا کنیم؟

پاسخ:
- Review Code با چک‌لیست SOLID
- مثال‌های خوب در تیم
- آموزش
- ابزارهای analysis static

۵.۳.۷ سؤال ۱۵: اگر کد قدیمی SOLID را نقض می‌کند، چه کار کنیم؟

پاسخ:
- Refactor تدریجی (نه یک‌جا)
- هر Sprint یک کلاس
- تست قبل از Refactor
- اولویت‌بندی بر اساس تغییرات آینده

۸ تمرین‌ها

۱.۸ تمرین ۱: تشخیص SRP

کد زیر را تحلیل کنید. چند مسئولیت دارد؟

```
{ reganaMtcudorP ssalc cilbup
} ... { )p tcudorP(tcudorPdda diov cilbup
} ... { )p tcudorP(tcudorPtide diov cilbup
} ... { )di tni(tcudorPeteled diov cilbup
```

```
} ... { )q gnirtS(stcudorPhcraes diov cilbup
    } ... { )(tropeRetareneg diov cilbup
                                }
```

شود. تقسیم کلاس ۵ به باید مسئولیت. ۵ راه‌حل:

۲.۸ تمرین ۲: Refactor با SOLID

کد زیر را با SOLID بازنویسی کنید:

```
                                { reganaMredrO ssalc cilbup
                            } ... { )o redrO(redrOetaerc diov cilbup
                             } ... { )di tni(redrOlecnac diov cilbup
                              } ... { )o redrO(liamEdnes diov cilbup
                           } ... { )o redrO(eciovnItnirp diov cilbup
                            } ... { )di tni(redrOkcart diov cilbup
                                }
```

OrderService، OrderRepository، EmailService، InvoiceService، TrackingService. به تقسیم راه‌حل:

۳.۸ تمرین ۳: تشخیص Coupling

در کد زیر، Coupling را تشخیص دهید:

```
                                { ecivreSresU ssalc cilbup
                            { )u resU(retsiger diov cilbup
  ;)(yrotisopeRresUsergtsoP wen = oper yrotisopeRresUsergtsoP
                                ;)u(evas.oper
        ;)(ecivreSliamEptmS wen = liame ecivreSliamEptmS
                            ;))(liamEteg.u(dnes.liame
                                }
                                }
```

به باید است. وابسته SmtpEmailService و PostgresUserRepository به UserService راه‌حل:
باشد. وابسته interface

۴.۸ تمرین ۴: Factory

یک PaymentFactory بنویسید که ۳ روش پرداخت (کارت، ،PayPal کریپتو) را پشتیبانی کند.

۵.۸ تمرین ۵: Strategy

یک ShippingStrategy بنویسید که ۳ روش ارسال (پست، تیپاکس، پیک) را محاسبه کند.

۹ منابع

۱.۹ منابع اصلی

منبع	فصل
Martin C. Robert — Code Clean	فصل ۱-۳
Martin C. Robert — Architecture Clean	فصل ۷-۱۰
Engineering Software — Sommerville	فصل ۷

۲.۹ منابع آنلاین

- ویدیو: SOLID Principles in ۱۰ minutes
- مقاله: The S.O.L.I.D. Principles
- کتاب رایگان: Refactoring.Guru

۱۰ چک‌لیست مطالعه

۱.۱۰ قبل از مطالعه

☐ جزوه را دانلود کردم
☐ وقت کافی دارم (۳-۴ ساعت)

۲.۱۰ در حین مطالعه

☐ بخش ۱ (مقدمه) را خواندم
☐ بخش ۲ (چهار اصل OOP) را خواندم
☐ بخش ۳ (SOLID) را خواندم
☐ بخش ۴ (SRP در HomeStore) را خواندم
☐ بخش ۵ (Coupling/Cohesion) را خواندم
☐ بخش ۶ (Design Patterns) را خواندم
☐ بخش ۷ (سؤالات) را خواندم
☐ بخش ۸ (تمرین‌ها) را خواندم

۳.۱۰ بعد از مطالعه

- ☐ سؤالات سطح ۱ را جواب دادم
- ☐ سؤالات سطح ۲ را جواب دادم
- ☐ سؤالات سطح ۳ را جواب دادم
- ☐ تمرین ۱ را انجام دادم
- ☐ تمرین ۲ را انجام دادم
- ☐ تمرین ۳ را انجام دادم
- ☐ تمرین ۴ را انجام دادم
- ☐ تمرین ۵ را انجام دادم

۱۱ جمع‌بندی

۱.۱۱ سه نکتهٔ کلیدی

۱. SOLID = پنج اصل برای کد قابل نگهداری
   - SRP: یک مسئولیت
   - OCP: باز برای توسعه
   - LSP: جایگزینی
   - ISP: interface کوچک
   - DIP: وابسته به انتزاع
۲. Coupling کم، Cohesion زیاد
   - Coupling: وابستگی بین کلاس‌ها
   - Cohesion: ارتباط داخلی کلاس
۳. Refactor یک عادت است
   - هر Sprint کمی
   - با تست
   - تدریجی

۲.۱۱ سؤال نهایی

اگر امروز کد Sprint ۱ خود را ببینید، چند تا کلاس SRP را نقض می‌کنند؟

پایان جزوه

موفق باشید!